Silvia Lucia Sanna[1], Cristina Alcaraz[1], Alessandro Sanna[1], Giorgio Giacinto[1], and Javier Lopez[1]

[1]Affiliation not available

June 18, 2025

MoLIFE: Methodology, Technologies, and Challenges for Mobile Live Intelligent Forensics Examination

Silvia Lucia Sanna, *Member IEEE*, Cristina Alcaraz, *Senior Member IEEE*, Alessandro Sanna, *Member IEEE*, Giorgio Giacinto, *Senior Member IEEE*, Javier Lopez, *Senior Member IEEE*

***Abstract*—Nowadays, mobile forensics is less explored in Digital Forensics case analysis due to the increase of protection mechanisms implemented by tech companies (i.e., Google for Android and Apple for iOS). As an example, the physical acquisition or the analysis of specific directories under super-user protection would corrupt the evidence. At the same time, the demand for mobile technologies and their respective communication systems is increasing exponentially, exposing numerous security threats and risks. For that reason, this paper presents MoLIFE, a novel Digital Forensics methodology for data acquisition and analysis of mobile devices. The proposed methodology is based on NIST standards for the Digital Forensics process, but adapted to new and emerging technologies by exploiting their power. MoLIFE can also be used to prevent cyber threats and incidents, as well as Digital Forensics post-mortem analysis, offering examples of applying the MoLIFE methodology and good practices for the future. To prove the technical feasibility of the methodology, a small case study on Android devices will be presented. As the methodology is based on new and emerging technologies, it depends on their limits that would be overcome in a few years.**

***Index Terms*—Mobile Forensics, Digital Forensics, Mobile Digital Twin, Artificial Intelligence.**

I. Introduction

MOBILE forensics is the science of retrieving data from mobile devices [1]. It includes a set of techniques to extract and analyze data stored in a mobile device when a specific crime is committed. Such analysis can be conducted only if requested and authorized by a judge. Nowadays, mobile devices are part of our everyday life, as well as tools for a wide variety of business-related tasks. In fact, they are widely employed for personal necessities or working activities (e.g., industries, critical infrastructure monitoring, healthcare data collection and monitoring, surveillance, etc.). Up to now, there are two main Operating Systems (OS) for mobile devices: Android, developed by Google and installed by many vendors (e.g., Samsung, Oppo, Xiaomi), and iOS, developed by Apple and installed in iPhones[1]. The main differences between such devices do not rely only on the OS used but also on the vendor [2]. In fact, different vendors can introduce distinct protection mechanisms, proprietary Software Applications (SwA), and custom hardware platforms. Reports show that there are 7.2 billion of mobile devices with millions of people connected worldwide [3], expected to grow with an investment of USD 6.79 Billion for 2029 [4]. The number of cyber attacks on mobile devices has thus increased rapidly in recent years [5].

Regarding cyber attacks, a Digital Forensics (DF) analysis is necessary for the incident response part, especially when dealing with a mobile device used in a business-related context such as an industry or company or related to critical infrastructures. A mobile device is often forensically analyzed even when a civil or penal crime is committed. Although the mobile device is not strictly related to the crime (i.e., it is not the first actor of the crime as in the case of a cybercrime), it is analyzed in the civil/penal trial because it can contain important information related to the case. For example, if a pseudo-pedopornography crime is committed, a mobile device could include pictures, chats, and website history, and for this reason, it must be analyzed. Due to the different vendors and proprietary systems, the analysis is not universal and relies strictly on them. Moreover, super-user permission (i.e., root, admin) is sometimes required to access specific data stored on the device. The acquisition of the super-user privilege strictly depends on the OS type and version, vendor type, and hardware specifics. In fact, most of the time, the mobile device is not analyzed because the acquisition of such permission compromises the entire device [6]. However, some DF tools such as `Ufed`[2], (i.e., an Israeli digital intelligence firm) and `Oxygen`[3] (i.e., a US-based investigative technology company) can dump a wide range of information even if the device is non-rooted, by using specific exploits to grant privileged access, but are becoming increasingly less effective due to security patches. As described by the NIST standard [7], one of the main principles of DF is the reliability of the data, including preservation of the state of the evidence (i.e., not corrupting the device and its data), the reproducibility and the availability of the data [8]. Hence, not acquiring super-user permissions properly sometimes leads to a data factory reset with a consequent data deletion, and such DF principle is violated. Therefore, the evidence cannot be analyzed and admitted in the trial. On the other hand, if the device is not analyzed, the case is partially solved from the legal perspective, as the device can contain essential data. Specifically, mobile devices are rarely physically analyzed , i.e., a bit-by-bit copy of the evidence at the seizure time to be fully replicated and

S. L. Sanna, A. Sanna and G. Giacinto are with the Dept of Electrical and Electronic Engineering, University of Cagliari, 09124 Cagliari, Italy e-mail: silvial.sanna@unica.it, alessandro.sanna96@unica.it, giorgio.giacinto@unica.it

S. L. Sanna is also with the Department of Computer, Control and Management Engineering, Sapienza University, Rome, Italy

G. Giacinto is also with the Interuniversity National Consortium for Informatics, CINI, Italy

C. Alcaraz and J. Lopez are with the Dept of Computer Science, University of Malaga, Campus de Teatinos s/n, 29071, Malaga, Spain, e-mail: alcaraz@uma.es, javierlopez@uma.es

Manuscript received MM DD, YYYY; revised MM DD, YYYY.

[1]https://gs.statcounter.com/os-market-share/mobile/worldwide

[2]https://cellebrite.com/en/ufed

[3]https://www.oxygenforensics.com

analyzed in every aspect. Sometimes, if the owner is willing to cooperate (i.e., a company in the case of a cyber incident or an innocent defendant), the secret message protecting the device is given, and a live forensics analysis of specific parts without super-user permission is made. In all other cases, modern forensics techniques must be developed explicitly for mobile investigation, such as bypassing security features and exploiting system vulnerabilities to read data [9]. Potentially, such methodology can lead to data integrity violations and should be supported by legislative frameworks.

To overcome these issues, this paper proposes a new methodology for Mobile Live Forensics (MoLIFE), based on the NIST Digital Forensics main principles, to analyze mobile devices even without acquiring super-user permissions on the real device. The NIST standard defines four main stages to be followed during a DF investigation: *(i) collection* to seize every device to be analyzed (i.e., everything with a memory, including both RAM or disk) without damaging and invalidating the evidence (i.e., do not physically break it) and taking pictures to prove it; *(ii) examination* to extract data from memory according to its type (i.e., differences between desktop and mobile, but also between OS types and different releases/versions) without damaging it and its data; *(iii) analysis* to objectively interpret the extracted data and find the important information; and *(iv) reporting* to write and deposit the report for the legal prosecution. The evidence must support every hypothesis to reconstruct what happened. Sometimes, this is difficult as the evidence belongs to a single instant instead of the whole incident time, as highlighted by Pierre Margot in "*Traceology, the bedrock of forensic science and its associated semantics*" [10]. Extracted data is prone to uncertainty and needs interpretation and correlation [11].

The MoLIFE methodology, depicted in Figure 1, can be applied in industries, companies, and critical infrastructures to monitor and prevent cyber incidents. Specifically, the MoLIFE methodology can support continuous monitoring and have complete certainty on what happened. The main pillar of MoLIFE methodology is a system of Digital Twins (DT) [12] with specific Artificial Intelligence (AI) models for automatic anomaly detection. The DT is a digital copy of the examined device in this case. The DT can be considered an emulator receiving real-time data from the physical device. Therefore, it replicates all its behavior but with super-user permissions and receives data in real-time from the physical device to be fully synchronized with the DT (i.e., being a complete copy). In fact, it is fundamental to create the DT systems before the device starts being used. This paper adapts the concept of DT to mobile devices; hence, the concept of mobile Digital Twin (mDT) is presented. The mDTs are integrated with specific AI algorithms to detect suspicious behaviors and anomalies, providing proper feedback to the real mobile device. Particularly, the methodology follows the three main stages of a security threat: *(i) before*, when the environment is clean, and no anomalies happened; *(ii) during*, while the threat is running to monitor the system and collect information; and *(iii) after*, to reconstruct the threat pipeline. An AI-supported mDT is assigned to each phase, communicating via TCP/IP, to comply with the bidirectionality introduced by Grieves *et al.* [13]. In the case of a cyber attack, this proposed system can then *(i) prevent* future incidents if based on known patterns and signatures; *(ii) detect* attacks in real-time and collect forensics information; and *(iii) reconstruct* the incident and understand whether it depends on the SwA behavior or user input.

The **main contribution** of this paper is thus the proposal of a new mobile live intelligent forensics methodology to acquire data that would typically require super-user access to prevent the failure of a mobile device, for example, in critical infrastructures. In this way, an anomaly is prevented. In case of a technical failure, the mDT and its integrated AI detection models could, for example, help with incident reconstruction. Another significant contribution is using different technologies in the mobile forensics field to overcome the problem of super-user permission requests. The mDT allows the analysis of a digital copy of the real device with the exact same content as the real physical device (usually not analyzed if there are no super-user privileges) without compromising the real device state. It is essential not to give the super-user permissions to the real device because it can be dangerous for the infrastructure, exposing the device to more security and privacy threats, as the malware or attacker immediately has super privileges to make their malicious actions. Moreover, this paper shows that the DF acquisition of an mDT has the same content as the real device, with the advantage of accessing admin data without compromising the real device.

The remainder of this paper is structured as follows. Section II describes the MoLIFE methodology, and Section III introduces all the technologies that could be integrated into the proposed MoLIFE. Section IV first describes some use case scenarios and then applies the MoLIFE to Android devices, the most used mobile device worldwide. The validity of the DF acquisition in Android mDT is discussed in Section V. The paper ends with the highlights on the open challenges and limitations driven by some current limitations in Section VI, whereas Section VII outlines the conclusions and future work.

II. Proposed Methodology - MoLIFE

This section presents the detailed methodology for the *(i) forensics prevention* stage (cf. Section II-B) to avoid the installation of specific SwA possibly associated with known threats and attacks; *(ii) live forensics* (cf. Section II-C) to detect at real-time the attack and collect forensics data; and *(iii) post-mortem forensics* (cf. Section II-D) to reconstruct the steps that lead to the incident starting from the forensics data acquired in the previous stages. The full procedure is described in the Diagram Flow depicted in Figure 1. MoLIFE follows the main pillars of DF designed by NIST [7], and each stage will respect them as shown in Table I.

A. MoLIFE Preamble

Each stage of MoLIFE is managed by a mobile DT (mDT), an emulated and virtualized mobile device with super-user privileges, interacting with the real device, having the same hardware characteristics and software logics (except a few cases as highlighted in Section IV-B, but leading to the same working mechanism and final results). The mDT, as

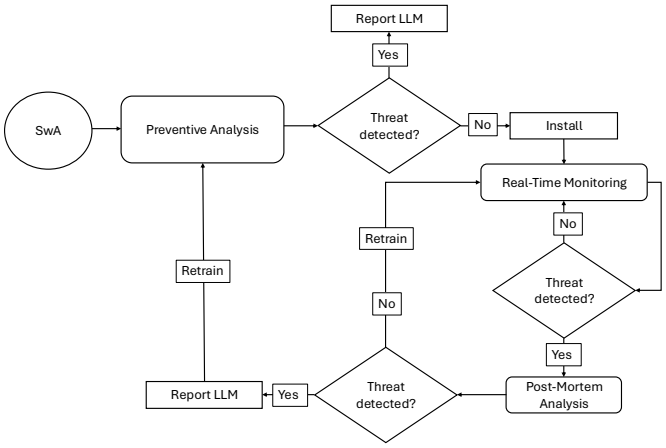

Fig. 1: MoLIFE flow diagram including all three mDT stages. Given a mobile SwA, first check if it is a known popular threat (preventive analysis) and, in case, report it. If the SwA is secure, it monitors its execution (real-time monitoring) and, once it detects the threat, reports the incident and finds the causes (post-mortem analysis)

a traditional DT, receives data from the real device in real-time (complete synchronization), processes them, and sends feedback to the real mobile device (bidirectionality principle). The communication is possible thanks to WiFi debugging, with the condition of having the mDTs and the real mobile device in the same network, for example, using a Virtual Private Network (VPN). Data elaboration on the mDT is integrated with AI algorithms to speed up the decisions, making them more accurate. The following subsections describe the two main technologies on which the MoLIFE is based.

1) ***Digital Twin****:* The DT is a digital virtualization of a real entity, completely synchronized with it, giving an advanced monitoring stage to the real system [14], [15]. A DT processes data as the real device, communicating with it and the sensors and actuators with multiple TCP/IP protocols. The DT is adopted to emulate, simulate, study, and prevent specific situations and behaviors on the real system without making changes and damages on the physical one [16]. The decisions and data processing could be guided by AI algorithms [17]. It is widely employed in the industry, critical infrastructures, Internet of Things (IoT), and Industrial IoT (IIoT) [18].

The mDT (i.e., a DT of a mobile device) can be applied in mobile digital forensics investigations in *(i) acquiring data* from the infrastructure by solving the super-user problem related to data loss (i.e., elevating the privileges before the synchronization); *(ii) preventing and detecting* real-time incidents by using highly computational costing algorithms; and *(iii) granting a backup and a collection* of detailed and specific data for each time in case of a future forensics analysis to understand how the incident happened. Nowadays, there is no DT for mobile devices, but some emulators have been released (e.g., Android Studio, Genymotion, AWS, XCode, etc.), allowing to run tests on a virtualized mobile device without the need of a physical mobile device. To turn these emulators into pure mDTs, data must be synchronized between the emulator and the real device through bidirectional communication. They should be in the same state simultaneously (or with a delay of very few seconds due to the latency), as in the IIoT. In the case of mobile devices, the synchronization could be done in two main ways: *(i)* using the WiFi debugging on the emulator and the real device, allowing to send and receive data over TCP/IP; and *(ii)* accessing to the same target SwA in the emulator and the real device with the same credentials and having data automatically synchronized via an external server managing the SwA (e.g., messaging apps, social networks and all the SwA with a cloud behind). The first presented solution is not that easy to implement. First of all, the input from the target SwA in the real device must be intercepted (i.e., needing an interceptor, such as Frida[4], to hook into a specific function of the target SwA and capture the input), then the input must be sent to the mDT via TCP/IP. Frida is a dynamic instrumentation toolkit for injecting scripts into running processes, enabling real-time analysis and manipulation across platforms. Additionally, it is necessary to modify the execution of the target SwA in the mDT to receive and process the captured input. At the same time, the execution of the target SwA must be stopped both in the real device and in the mDT and restored once the input is processed. The procedure is not easy and challenging because it causes a loss of time, meaning a desynchronization. Moreover, this technique needs a repacking on the target SwA, creating two versions from the original: one in the real device for the non-rooted interception mechanism with *frida-gadget* and the second for the mDT to receive the input properly. Repacking a SwA is sometimes impossible due to anti-tampering techniques to prevent reverse-engineering and software cloning [19]. In some cases, such as for memory forensics, data cannot be injected in the RAM. The same holds for permanent storage, which can be simply copied and pasted. For this reason, it should be better to have the same access of the target SwA in the mDT and the real device, allowing automatic data synchronization, only focusing on behavior interception on the mDT and sending a stop command with the WiFi debugging to the real device in case of a failure. The use of an mDT is more helpful than the analysis of the mobile data stored in the cloud because it allows to *(i) analyze the full SwA* behavior and not only dump its content; *(ii) analyze the effects* of the target SwA in the system; *(iii) monitor the target SwA*, above all even without root access; and *(iv) access its data immediately* without waiting another jurisdiction to answer and consent the analysis of the data stored in the cloud of another country with different legislation and data management. Additionally, the mDT helps adding computationally expensive tasks that the real mobile device could not support. An example is the case with AI detection models or other identification mechanisms.

2) ***Artificial Intelligence****:* MoLIFE system uses AI to detect suspicious threats and behaviors while accurately classifying them. Due to the lack of training data, semi-supervised Deep Learning (DL) models are preferred. Moreover, for each decision taken, it is important to use explainable AI

[4]https://frida.re/

(xAI), i.e., methods and techniques in AI that make the behavior and decisions of AI systems understandable and interpretable to humans, especially in complex models like deep learning. xAI is also used to understand which features influenced the detection. Additionally, external CTI sources are interpreted by Large Language Models (LLMs) and Natural Language Processing (NLP), as they can quickly extract key information, summarize complex content, and identify patterns or anomalies across large volumes of unstructured text with minimal human intervention. At the same time, they can produce human-readable text given structured or unstructured data and follow the known grammar and syntax rules. LLMs are advanced AI systems trained on vast amounts of text data to understand, generate, and interact with human language contextually and meaningfully. NLP are AI models that understand, interpret, generate, and respond to human language in a meaningful way. These algorithms are also helpful in generating reports for the preventive and post-mortem stages to the human analyst. Because of the high computational power needed by these algorithms and corresponding energy consumption (i.e., battery drain), it is better to run them on an mDT having the same characteristics as the mobile OS but with the highest resources thanks to the server.

B. Stage 1: Preventive Forensics

This stage is necessary to avoid installing a specific SwA containing known signatures belonging to malware or a possible threat. As the name suggests, it is designed to prevent a cyber incident. When the user wants to install a new SwA in the real device, it sends a request to the mDT associated with this stage. Thus, the mDT analyzes the SwA accordingly and, if no anomaly is detected, the SwA can be installed on the physical device.

The analysis is mainly made up of two stages. A first one was made on the ***hash matching*** to prevent the installation of an unknown and unofficial SwA. A SwA could be repacked to add exploitation or malware behavior, or retrieve important owner data and change the apparently official SwA [20]. Repacking is very popular [21], [22], and cracks are widely common, above all, when a SwA has payment services (e.g., Spotify premium). The repacked SwA contains the same features as the original, and the user cannot recognize it during execution because it has the same characteristics and functionalities, with only hidden malicious behavior [20]. Of course, the SwA has a different hash than the original one but no match with ssdeep, feature hash, permhash. Because of this, the hash detector can check if the requested SwA is unofficial and an indicator of a repacked app with potentially malicious behaviors. Some techniques on code similarities can be used for the detection [23]. The hash detector is based on a simple database containing the hashes of the known, legitimate, and interesting apps and those of the previous versions so that they can be easily installed. The database is stored on a server. Different Cyber Threat Intelligence (CTI) online systems can be queried on the computed SwA's hash to improve security. These systems (e.g., Virus Total, MITRE CVE, NIST NVD), have a report and information on many SwA stored by hash. Their analysis can be integrated into the MoLIFE using LLMs and NLP algorithms.

If the check of the hash value is passed, the app is analyzed by a ***pre-trained AI*** model to check specific static and dynamic features related to known vulnerabilities and malware behaviors [24]. This is the second internal stage. Most of the current literature approaches for malware analysis and detection are based on *static* analysis, which is code inspection without executing the SwA and highlighting the presence of specific patterns, e.g., suspicious functions, API calls, IP addresses, cleartext strings, etc. For better runtime detection, dynamic analysis is needed, consisting of running the SwA in a protected but real environment called a sandbox (i.e., controlled, isolated environment used in computing to run, test, or analyze code without risking harm to the host system or data) and checking its execution, (e.g., monitoring the function calls, sometimes even with hooking and interception, observing the state of the RAM, inspecting the network traffic analysis, etc.). With the use of AI in cybersecurity, some algorithms have been adapted to detect known vulnerabilities and malware behaviors starting from specific, explicit and popular features (both static and dynamic) [25], [26]. These algorithms must be *robust to adversarial attacks* to avoid the misclassification of a sample modified by the attacker to keep its malicious behavior and evade the AI detection algorithm [27]. Adversarial attacks are perturbations applied to input data, crafted with the specific intent of causing a Machine Learning model to produce incorrect or suboptimal outputs, e.g., classifying the malicious sample as the benign one. These perturbations are often imperceptible to humans but exploit vulnerabilities in the model's decision boundaries, thereby exposing limitations in its generalization and robustness. This is also possible because sometimes the selected features are not robust enough, and the system can be easily deceived [28]. Indeed, adversarial attacks in this preventive forensics stage are very dangerous because if a malicious application is not detected, it will be installed in the real device and communicate with the critical infrastructure or scenario to be protected. This leads to a specific threat and consequent incident. The AI algorithm must consider the temporal drift and different malware families [29], [30] to better recognize new threats and variants.

When no threat is detected, the SwA can be installed in the mDT of the second stage (*Live Forensics*) and subsequently in the real device. The state of the SwA before installation is called $state_0$. If the algorithm raises security threat alarms, the SwA is not installed. A report is generated for the human operator using LLMs. A clean backup copy without the traces of the conducted analysis is restored after every analysis so as not to influence future detections. To understand why the SwA cannot be installed, it is important to know which are the features that influenced the AI model in the decision of the "vulnerable/malicious" class. For this reason, techniques of xAI [31] are applied.

According to the NIST standard, this stage follows all the phases: the (i) *collection* phase to identify data from the running application; the *(ii) examination* because it extracts potentially risky features from the application; regarding the

(iii) analysis phase, the system automatically interprets and takes a decision on the extracted data with a conjunction analysis from external CTI tools and in case of a detected suspicious behaviors, *(iv) report* it to the human analyst.

Algorithm 1 MoLIFE Prevention Algorithm

Require: SwA, hash database, CTI reports
Ensure: Pre-trained models on static and dynamic features
1: Hash Match
2: CTI-driven Analysis
3: Extract static and dynamic features
4: **return** Classification
5: **output** Report with LLMs

C. Stage 2: Live Forensics

Sometimes malware reveals its malicious behavior after an elapsed time or event, or a vulnerability is exploited when someone discovers it or by specific user input [32], [33]. Moreover, the analysis is fast due to the short time a SwA can be kept in "quarantine" because the user needs it, especially in critical scenarios. For this reason, the preventive forensics stage is not sufficient in itself. Accordingly, MoLIFE needs live forensics analysis to continuously monitor the SwA and its effects on the device at runtime and during standard execution. The live forensics stage supervises and collects forensics data in case of future possible DF analysis with a more forensically oriented algorithm. For example, the full RAM content can be analyzed to check the effects of a specific target SwA in the whole system at runtime. In addition, the target SwA can be inspected with instrumentation tools (e.g., Frida) to check specific behaviors at run-time. The network connections and communications can be inspected to check which IPs and domains the SwA contacts and retrieve the communication payload. As the mDT has super-user privileges, specific directories can be monitored to check if uncommon and potentially dangerous files are downloaded, created, or accessed by the SwA at runtime and stored in the device. It is worth pointing out that this simulation technology is potentially dangerous, as the mDT includes intellectual property and personal data, whose disclosure could cause significant consequences and irreparable damages [34], [35]. In this scenario, the mDT can check if specific, unauthorized, illegitimate, and uncommon data is read and exploited by the SwA outside of its classic, standard, and declared behavior obtained from the previous preventive forensics stage investigation. The ***forensics-based AI model*** used must be robust to adversarial attacks to avoid an attacker using anti-forensics techniques [36], [37] and steganography (i.e., hiding information in other sources such as images, audio, video, text, to not be easily detected) techniques [20], [38] to hide behaviors and bypass detection. Even in this stage, the AI algorithm includes xAI-based techniques to find the features that have the highest influence in classifying the behavior as a security threat, and the related reason. This is important for the next post-mortem analysis stage to reconstruct what happened. In case of a cyber attack, the SwA is blocked first on the real device to prevent more damage, avoiding desynchronization, and, lately, on the mDT. Subsequently, the collected data are sent to the post-mortem forensics to reconstruct what happened automatically.

This stage follows the NIST standard in the *(i) collection* phase when it identifies the forensics data to which to pay attention during the SwA execution; the *(ii) examination* phase by extracting information and features from the identified forensics data; the *(iii) analysis* to interpret the forensics data under attention. This phase does not have a traditional *(iv) reporting* phase because no final report is generated for the human analyst, but in some sense, sending the features that determined the threat classification and the collected data to the next stage could be considered a sort of report as the information is described.

Algorithm 2 MoLIFE Live Algorithm

Require: SwA
Ensure: Pre-trained models live forensics dynamic features
1: Network Traffic Analysis
2: RAM Monitoring
3: Disk File Analysis (content and creation)
4: Explain the classification (xAI)
5: **return** Live Detection Classification
6: **output** xAI, collected forensics data

D. Stage 3: Post-Mortem Forensics

To understand whether the threat detected has been raised by user input or hidden behavior of the SwA, a third stage is required to investigate what happened, i.e., post-mortem analysis. To start the analysis, the post-mortem forensics analysis needs the original data at the time of installation from the preventive stage (i.e., SwA in the $state_0$ after the analysis and before installation). Additionally, it receives from the live stage its conducted analysis, i.e., the incident information with the actual state of the SwA, the forensics data and the xAI features that lead to the alarm of a suspicious behavior. At this stage, the mDT continuously sends specific inputs to the original SwA and stops when it reaches the state of the SwA at the moment of the attack (i.e., the state of the SwA recovered from the live forensics stage). This technique is called ***fuzzing*** [39], [40], and it is made of different methods to generate a set of inputs on specific software and SwA aimed at detecting vulnerabilities, bugs or anomalous behaviors. Random input is generated from a starting set, usually defined by the user according to the specific SwA. This input is sometimes managed by AI algorithms such as LLMs [41] as they can produce syntactically valid and semantically rich test cases tailored to the target program's structure. The program is executed each time with a single input, and its behavior and output are tracked. No vulnerability or security issue is reported if the output is the same as the expected one. In contrast, if some crash, error, or anomaly is detected, the fuzzing program reports it to the human analyst, logging the input that produced it. In the literature, there are three different types of fuzzing: *(i) black-box*, when the tests are made on external input and the fuzzer does not know the program's

structure under test, and general analysis is made; *(ii) white-box*, fuzzing when the program source code and structure are available, i.e., specific test cases are generated, and the analysis is specific; and *(iii) gray-box*, fuzzing to use a restricted set of internal program's knowledge for the tests.

Fuzzing is used in MoLIFE to determine what led to the threat and reconstruct how it happened. In particular, if the threat has been raised by user input (and which one), or if the real hidden behavior of the SwA emerged after a specific elapsed time or event. As the structure of the target SwA is known from the analysis conducted in the preventive stage, white-box fuzzing is preferred with inputs generated by specific LLMs for wider coverage. Moreover, the fuzzing procedure can be led by an AI model that previously explored the structure of the target SwA from the information collected in the preventive forensics stage and its prior knowledge. At this stage, as for preventive forensics, the data collected for the analysis are also analyzed through online and reliable CTI tools. Their output is processed via LLMs, not influencing the algorithm's decision but just giving more details and data on which to base the output; they are only additional information to enrich the decision and improve the final report. In the case the fuzzing algorithm does not detect the threat, this means that the AI model in the live forensics stage made some error (and this can happen because of insufficient training data or for anti-forensics, steganography, anti-analysis techniques used by the malware [42]). Hence, the mDT will send a notification to continue the execution first on the live forensics mDT and then on the real device after the AI model of the live stage is retrained. Otherwise, the same incident is reported. On the other hand, if a cyber-attack has been detected, the execution remains blocked, and a report with LLMs is generated for the external forensics human analyst to know what happened, make a manual analysis, and make a correct decision.

This last stage follows the NIST standard in the *(i) collection* because it receives the data from the preventive forensics stage (i.e., the original SwA and its analysis). The live forensics system (i.e., the state of the SwA at the moment of the incident, the xAI features) and from external CTI tools. The NIST *(ii) examination* phase is used to interpret all the received data, the original SwA, and the incident state. The *(iii) analysis* phase corresponds to the check at each step of what happened and how the incident occurred by using the fuzzing techniques. When the incident state is reached, or after many analyses, no threat is detected, the system gives in output a *(iv) report* of the findings to the human forensics analyst.

Algorithm 3 MoLIFE Post-Mortem Algorithm

Require: SwA in $state_0$, Live Features, xAI Live Analysis, CTI reports,
Ensure: Pre-trained AI-based Fuzzing
1: Fuzz the SwA in $state_0$
2: Stop when reaching the same forensics data of Live Analysis
3: **return** Decision on retraining previous algorithms based on new features and findings
4: **output** Report the incident's causes

III. Support Technologies for MoLIFE

MoLIFE needs some technologies to work, such as a communication system, AI algorithms, and a powerful machine to run the emulator for the mDT. For this reason, the following Section highlights how the technologies can be applied to MoLIFE. The selected technologies must comply with the cybersecurity principles such as the **CIA-triad** i.e., limiting data access (confidentiality), ensures accurate data (integrity) and makes them always accessible (availability); **resilience** to cyber attacks, robustness, avoiding a failure and collapse of the system; and **non-repudiation** integrity and authenticity. They are represented in Table I.

MoLIFE needs a fast **communication** system to ensure data synchronization between the physical device and the emulated one. Cloud, Edge, Hybrid, and Quantum Computing ensure this. The first solution offers high storage and processing power but requires high energy consumption and high latency when accessing the data. For this last reason, the Cloud is inefficient in some scenarios, and hence, the Edge solution is preferred as it ensures the nodes' high mobility is closer to the end user. An intermediate solution is the use of Hybrid Computing, i.e., a combination of multiple types of computing systems or architectures to optimize performance, efficiency, and flexibility. Hence, Hybrid Computing allows organizations to combine the scalability of the public Cloud with the security and control of private infrastructure, enabling flexible, efficient, and secure computing environments. Quantum Computing leverages the principles of quantum mechanics (e.g., superposition and entanglement) to solve complex problems significantly faster than classical computing systems. It allows fast communication, fast data processing, and better performance of AI algorithms. For these reasons, quantum computing is a good option because when it is available, we will have the fastest executions. However, at the time of writing this article, the technology is not ready to be used, but promising studies affirm its availability in the future [43]. One first step for fast communication relies on a good implementation of different technologies for mobile communication (e.g., WiFi, Bluetooth, 5/6G) supported and needed by the mDT for data synchronization.

As MoLIFE must comply with the CIA-triad, the use of blockchain technology can help. Blockchain technology is a decentralized distributed digital ledger that records data exchange (i.e., transaction) between multiple entities to ensure security, transparency, and immutability. Generally, it is used for cryptocurrency transactions, smart contracts, supply chain management, and voting systems. Blockchain technology tracks every data exchange between nodes by giving a unique hash to the nodes and transactions. Therefore, blockchain technology can track data inconsistency, maintaining the principle of data integrity in Digital Forensics. Hence, by design, the blockchain technology proves data integrity and non-modification, a fundamental principle of MoLIFE. Other systems, such as mobile communication systems, AI algorithms, mDT, and computing systems, must be implemented correctly to ensure data integrity. For example, the communication system must not lose any transmitted data and guarantee that

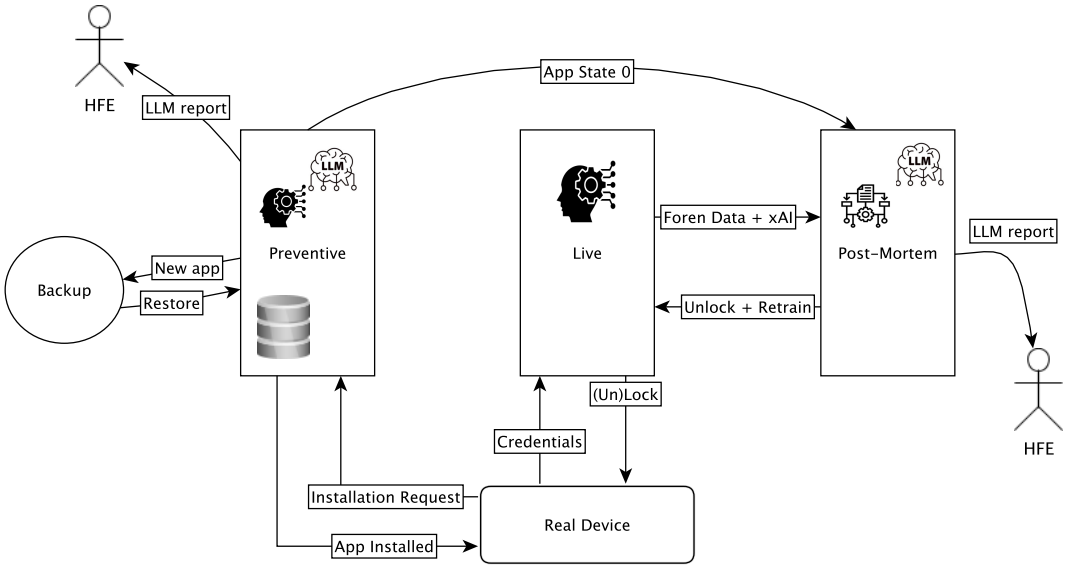

Fig. 2: Methodology schema. The three stages *Preventive* with the *clean backup*, *Live*, and *Post-Mortem* are presented with the interaction between them. The *Preventive* stage receives the installation request from the real device, analyzes it, and installs the application in case of no anomalies, restores the clean state, and generates the report. In case of an anomaly, the *Live* stage receives the login credentials, locks the real device, and sends forensics data and xAI output to the post-mortem stage. The *Post-Mortem* stage receives the forensics incident data, the xAI, the app's original state, and, in case no anomaly is detected, sends an unlocking signal to the live stage to be redirected to the real device and generates the report with LLMs.

no one can intercept and modify it, i.e., resilient to sniffing attacks. AI guarantees data integrity with an accurate and consistent prediction, i.e., samples are correctly classified and do not change with the different systems used. At the same time, data integrity is ensured by mDT and computing systems by guaranteeing that no one can change the stored data. For example, different solutions such as anti-viruses, access control, action logging, and firewalls can be implemented.

Due to data decentralization, the blockchain guarantees data availability every time. By design, data availability is also ensured by Edge and Hybrid Computing by combining the strengths of local processing with the scalability of cloud resources. Quantum computing does not directly guarantee data availability, but it has enormous potential to optimize, secure, and support the infrastructure that will ensure data availability in the future. Data availability in AI refers to ensuring that high-quality, relevant, and timely data is accessible whenever AI systems need it for training, inference, or continuous learning. Without strong data availability, AI systems degrade in performance, become biased, or even fail outright. Hence, AI algorithms must be implemented in this way. A good implementation of Mobile Communication systems with 5/6G, WiFi, and Bluetooth can guarantee data availability, i.e., data can be transmitted, accessed, and received at any time. For example, implementing network redundancy, adaptive routing, load balancing, and Quality of Service mechanisms.

The blockchain guarantees confidentiality by default thanks to the encryption implemented in the stored chain, and only parties with the decryption keys can read the data. Due to the use of smart contracts and trusted execution environments, data is processed privately. Other systems can guarantee the confidentiality principle with a correct implementation. For example, in AI, the model must not use private data for the training; they must be anonymized, and no one can infer the training dataset in any way. The mDT and computing systems must be implemented with access control logic to ensure data confidentiality.

The CIA-triad is strictly related to the resilience principle. MoLIFE and its supported technologies must be designed to be robust to cyberattacks. By design, it is ensured by the consensus mechanism implemented in the blockchain and its decentralization design (i.e., without a central point of failure). The other technologies must be implemented correctly to be resilient. For example, AI algorithms must be robust to adversarial and presentation attacks so that no one can infer the model to know how it decides its training dataset and modify the sample to bypass the detection (i.e., a malicious software not detected by the system). At the same time, the mDT and the computing systems must implement standard measures to be robust to cyberattacks, such as using robust IDS and robust anti-viruses.

The non-repudiation principle is necessary by MoLIFE to demonstrate who accessed the forensics data, what they did, and how they modified it. This principle is ensured by the blockchain's design, which can keep track of the transaction and data modification on the node because of the decentralized consensus, ledger immutability, and cryptography. The communication systems also ensure the principle by design because every connection is identified by specific data such as IP addresses, MAC addresses, phone numbers, and

		Stages			Requirements					
		Preventive	Live	Post-Mortem	Communication	CIA			Resilience	Non-Repudiation
						Confidentiality	Integrity	Availability		
	mDT	✓	✓	✓	○	○	○	×	○	×
	AI algorithms	✓	✓	✓	○	–	○	○	○	–
Technologies	Mobile Communication	✓	✓	✓	○	○	○	○	○	✓
	Cloud Computing	✓	–	✓	×	○	○	×	○	○
	Edge Computing	✓	✓	✓	✓	○	○	✓	○	○
	Hybrid Computing	✓	✓	✓	✓	○	○	✓	○	○
	Blockchain	✓	✓	✓	–	✓	✓	✓	✓	✓
	Quantum Computing	✓	✓	✓	✓	○	○	✓	○	○
NIST DF	Collection	✓	✓	✓	–	–	–	–	–	–
	Examination	✓	✓	✓	–	–	–	–	–	–
	Analysis	✓	✓	✓	–	–	–	–	–	–
	Reporting	✓	–	✓	–	–	–	–	–	–

TABLE I: MoLIFE stages, technologies, forensics. The table shows the technologies adopted for each MoLIFE stage and their security requirements. The last lines of the table show the 4 main principles of DF designed by NIST and respected in the 3 MoLIFE stages.

Legend

✓	The technology **satisfies by design** the requirement **essential** to MoLIFE
–	The technology **dissatisfies by design** the requirement **unnecessary** to MoLIFE
×	The technology **dissatisfies by design** the requirement **essential** to MoLIFE
○	The technology **requires proper setup** for the requirement **essential** to MoLIFE
	MoLIFE main **essential** technologies

Bluetooth addresses. The AI algorithms are not designed to guarantee non-repudiation; up to now, it does not depend on a good implementation. The mDT and computing systems can guarantee non-repudiation with proper access control.

IV. MoLIFE-based Scenarios

This section lists one of the MoLIFE possible application scenarios. Section IV-B presents the digital forensics acquisition from an Android mDT, the most used mobile operating system worldwide.

A. Application Scenarios

The MoLIFE system has been designed for those specific industries classified as critical infrastructures, where it is important to protect a human operator, and specific data or areas of the industrial plant while using a smartphone device. A human worker has a specific smartphone device used for job activities and that can be used only in a particular area with a VPN. The mobile device can access specific industrial data with which the human operator works. Such systems are called Mobile Device Management (MDM) [44]. In most cases, the companies rely on third-party applications to monitor the job-mobile devices, relying on third parties for security. Such applications could contain some vulnerabilities that can be exploited if the device or the company infrastructure is not protected. Instead, with this system of mDTs, the protection directly depends on and relies on the company. In MoLIFE, the given mobile device is lightly modified to interact with the system of mDTs, *(i) prevent* the installation of specific SwA unless the preventive forensics stage sends a granting signal (i.e., deny SwA installation unless authorized); *(ii) synchronize* with the mDT in the live forensics stage by sending data automatically and in live mode; and *(iii) block* a specific SwA execution and resume it when an appropriate signal is received. This smartphone can protect the user and the company's sensitive data (preventing industrial cyber espionage) thanks to the mDT system.

B. DF Acquisition of Android DT

This section presents a case study of the system adapted and based on Android devices, the most common mobile OS. This means that they are more exposed to cyber attacks, vulnerabilities, malware, and every form of threat, as detailed in Section I. Moreover, it is the cheapest device due to its limited cost compared to Apple mobile devices (iOS) due to the Asian companies such as Oppo, Xiaomi, OnePlus, Samsung, etc, which produce them highly. For this reason, Android devices are commonly used by companies, public organizations, and entities. Android is an open-source OS, i.e., meaning it is available to anyone, and can be tested by the community. This fact allows analyzing applications to check if they exploit OS vulnerabilities, thus exhibiting malicious activities as detailed in previous sections. Due to these factors, mobile cybersecurity is more centered on Android devices. This does not mean that iOS is not analyzed, but as it is proprietary, more closed, and less massively used, fewer studies have been developed on it.

This section presents a case study on the validity of an Android forensics acquisition in an mDT that produces the same result as acquiring a real rooted Android device. In detail, it explains the different acquisition and analysis methodologies to be followed according to the data to be analyzed and the difference between the mDT and real device. Because of the various architectures on which the real device and the mDT (some internals cannot be emulated) run, sometimes the acquisition methodology can change. As a result, the extracted content and analysis can also differ slightly. It will be detailed in Section V. The following subsections highlight how to acquire data in the mDT, starting with the methodology used on the real device. Sometimes, the mDT is called a

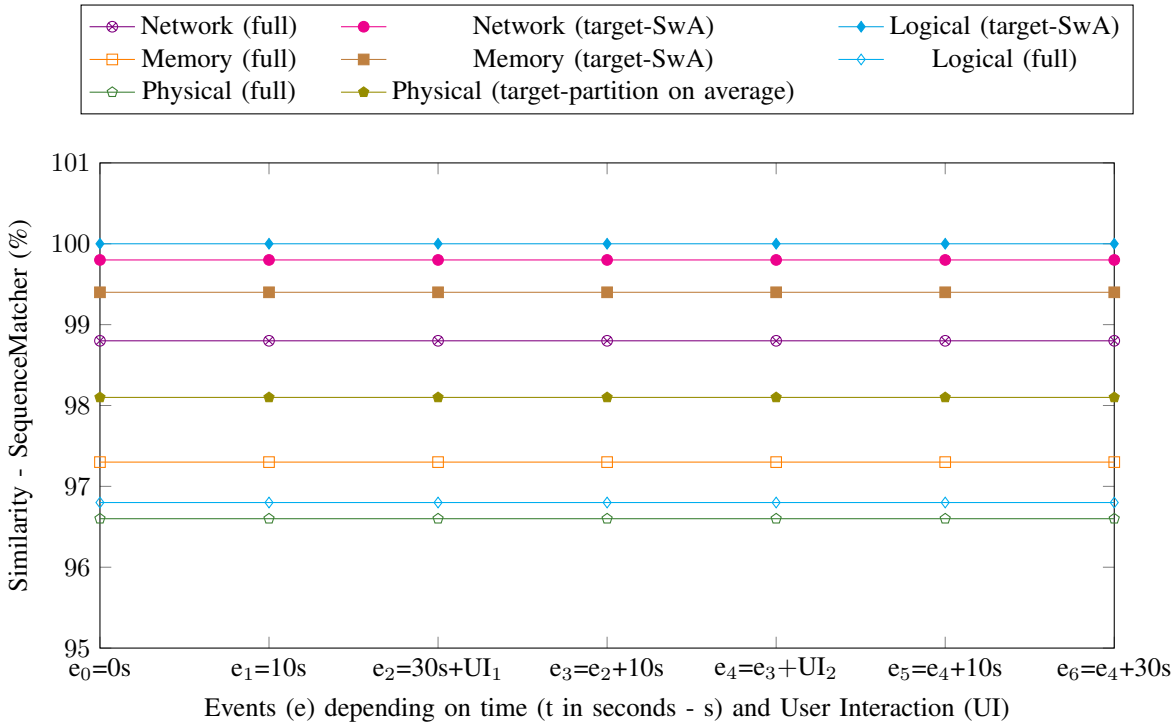

Fig. 3: Acquisition percentages over time (t) and user interaction (UI). The graph shows, for the 8 different forensics acquisition methodologies, the similarities between the acquired analyzed data between the mDT and the real device under the same running condition (same architecture, OS, running application and version, running time, and user inputs). Similarity has been computed with the *Ratcliff/Obershelp* algorithm

mere emulator when considering only its structure and the digital copy of the real device, its emulation. Hence, it does not depend on the capability of receiving, elaborating, and sending data in real-time synchronized with the real device (the definition of the emulator being called mDT).

Network Acquisition and Analysis: monitors the network traffic by recording and saving in a file format called *pcap*, the exchanged packets at every protocol level and using different protocols. For both systems (the real and the mDT), the tool `tcpdump` is used for the acquisition, and tools, e.g., `Wireshark`, `tshark`, `Zeek`, Python libraries e.g., `Scapy` can be used for the analysis of the saved pcap file. Target network analysis can be done with tools like `pcapdroid`[5] but also with `tcpdump` selecting the UID of the target application found with `iptables`, saving the content in a pcap file analyzed as the full network dumps.

Memory Acquisition and Analysis: referred to as the RAM analysis, this analysis represents the most challenging one because, due to the nature of the RAM, the content is highly volatile and unstable, it changes over time as the system (e.g., by using the garbage collector) inserts the content of a variable in the memory only when needed, and de-allocates it when it is no longer necessary. The memory acquisition can be made *(i)* at the level of a target application, i.e., extracting only the content of the RAM allocated to that specific application; and *(ii)* dumping the complete RAM to monitor the whole system in the RAM and track the effects on the system from the main memory after a specific behavior. The first one can be done with tools like `Fridump`[6] based on the instrumentation of the app via `Frida`, analyzed with strings or simple hexadecimal view or specific encodings (e.g., strings, images, audio, etc.). On the other hand, the complete acquisition can be made with `LiME`[7]. In this case, an extra step is needed, with the correct compilation of the kernel, to get a map of the memory and understand how the OS manages the memory and its content. The kernel can be compiled both in the emulated device and in the real device following the same procedure. The extracted content of the RAM called *image of the RAM* can be analyzed with the tool `Volatility`[8].

Logical Acquisition and Analysis: means the extraction of all files and directories accessible by the user (e.g., those belonging to each user, application, or system) that is done as a copy of the non-deleted content. As for the case of RAM analysis, this can be done only for a specific target application or for the full file system. To acquire it, is it sufficient to copy the directory under root permissions in a directory without root (e.g., SD card) so that the content can be downloaded via `adb pull` command (accessible only for directories not under the super-user privileges). At the same time, files and directories not protected by super-user privileges can be dumped on a local PC without any transfer on the SD card. The analysis can

[5]https://github.com/emanuele-f/PCAPdroid

[6]https://github.com/Nightbringer21/fridump

[7]https://github.com/504ensicsLabs/LiME

[8]https://github.com/volatilityfoundation/volatility

be done for both systems with text editor tools or by analyzing the hexadecimal structure and binary similarities techniques.

Physical Acquisition and Analysis: concerns a complete bit-by-bit copy of the evidence under analysis, able to retrieve even the deleted content not overwritten in memory or retrieve a footprint from the filesystem. Forensically, this is the best acquisition methodology because, due to the perfect bit-by-bit copy, the analysis would be the same as done in the evidence but could be repeatable (one of the forensics principles). In a real device, the bit-by-bit acquisition can be done with specialized commercial tools such as `Ufed`, which cannot be used in an emulator due to the specifics of UFED. A free alternative in the real device, that does not required a paid license, is the acquisition of the *mmcblk0* using the *dd* command `adb exec-out su -c "dd if=/dev/block/mmcblk0" > memdump.img`. However, this partition cannot be found on the emulated device. The partition of the mDT strictly depends on the emulator tool (e.g., Android Studio[9] and Genymotion[10]) and on the selected device and OS version. In the emulated device, the physical acquisition can be done with the same `dd` command but considering the *loop* and *sda* partitions in the */dev/block* path. Each of them contains data about specific functionalities, such as application data, system data, etc. For this reason, to have the complete dump, all of them must be copied; otherwise, the analysis must be conducted only on the interesting target SwA.

V. Experimental results and discussion

This section shows the results of the forensics acquisition methodology described in Section IV-B, highlighting what is the difference in the analysis between the dumped content from a real Android-rooted device and its mDT. Tests were done using Android Studio emulated with an ARM 64 chipset as the real device (using a Macbook M3 computer). The real device and the mDT run with the same architecture (i.e., arm64v8a) and with the same OS and SDK version (i.e., Android 14, API 34). To demonstrate the consistency of the results, the experiments were repeated for 10, following the same conditions to prove the system's stability. The same SwA with the same version has been executed simultaneously on both devices, with the same login and same interactions to guarantee data synchronization. Simultaneous interaction is possible with `adb` when specifying the device with the option *-s DEV_NAME*. Data has been sent and intercepted with `Frida`[11].

Figure 3 shows that data acquired from the mDT and the real device are quite similar. The content is unchanged between them, but sometimes we could not have the 100% similarity or the same hash because of the emulator, as described in Section II-A1 and Section IV-B. Some vendor devices are not emulated by tools like Android Studio and Genymotion, e.g., specific internals of *Samsung*. Notably, such data are emulated by other mechanisms in the mDT. For example, in a real Samsung device the contacts are managed by Samsung itself (as shown in Listing 1). In contrast, in an emulated device, they are managed by the emulator as shown for Genymotion emulating Samsung Galaxy S4 in Listing 2 and Android Studio emulating Google Pixel 9 in Listing 3. Some other apps developed by the vendors can be downloaded and installed accordingly in the mDT and subsequently accessed with the same login data for synchronization. At the same time, the emulator contains proprietary data for the mDT belonging to Genymotion or Android Studio, according to which emulator is used. For example, such directories contain data on the communication between the emulator and the Genymotion control system (i.e., com.genymotion.genyd) or preferences and configuration options set by the user or system (i.e., com.genymotion.settings). Data that cannot be emulated do not contain interesting information for the forensics investigation. Additionally when talking about internals to manage the pure emulation process.

Listing 1: Listing of contact-related packages in /data/data directory for Samsung Galaxy S4 real device

```
a33x:/data/data # ls | grep "contacts"
com.samsung.android.app.contacts
com.samsung.android.providers.contacts
com.sec.android.widgetapp.easymodecontactswidget
```

Listing 2: Listing of contact-related packages in /data/data directory for emulated Samsung Galaxy S4 with Genymotion

```
a33x:/data/data # ls | grep "contacts"
com.android.contacts
com.android.providers.contacts
```

Listing 3: Listing of contact-related packages in /data/data directory for Pixel 9 API 35 (emu64a)

```
emu64a:/data/data # ls | grep "contacts"
com.android.providers.contacts
com.android.providers.contacts.
auto_generated_rro_product__
com.google.android.contacts
```

For this reason, the emulators do not manage SwA and services developed by the vendor that are always found in a real device. On the other hand, an emulator has additional services and SwA managing the emulator itself (i.e., directories with the name *androidstudio* or *genymotion* according to the used emulator). Such directories cannot be deleted because they are essential for the mDT to work and emulate Android internals. Despite this, they do not affect the acquisition of forensics in terms of data integrity and data content (i.e., user data and effects of the apps in the whole system). Other differences rely on the memory offset and network connection management while the data content is exactly the same. These are not limitations, as the emulated device contains the same data as the real device when a specific amount of time or event happens (as illustrated in Figure 3). At the same time, the management of the emulation process does not affect these actions, and the related directories do not contain data that is useful for forensic investigation.

The `SHA256` hashing algorithm is commonly used in DF to ensure file integrity, as it provides collision-resistant identification of data. Ideally, hashes verify that forensic acquisitions

[9]https://developer.android.com/studio
[10]https://www.genymotion.com/
[11]https://frida.re/docs/android/

from mDTs precisely match real devices. However, in practice, identical hashes are rarely achieved due to subtle differences in directory structures, hardware architectures (ARM vs. Intel), timing discrepancies, and memory management variations between real Android devices and their emulated counterparts. Despite differing hashes, mDTs reliably replicate real devices' behavior, memory content, file structures, and API interactions. Traditional comparison methods like `ssdeep` hashes and Linux's `diff` are ineffective because of architecture-induced offsets. Instead, the Ratcliff/Obershelp[12] algorithm implemented in Python's *SequenceMatcher* of `difflib`[13] is recommended for assessing content similarity. This method calculates a similarity ratio based on matching characters, typically yielding over 95% similarity, effectively confirming content equivalence despite minor structural or temporal discrepancies. The Ratcliff/Obershelp similarity ratio R between two sequences A and B is defined as:

$$R = \frac{2M}{|A| + |B|} \tag{1}$$

where M is the total number of matching characters in the longest common subsequences found recursively, and $|A|$, $|B|$ are the lengths of the two sequences. This ratio effectively measures binary similarity, especially when structural variations exist but content remains functionally equivalent.

The forensics analysis on the mDT allows to monitor and reconstruct the full activity of the user, the effects of an application in the system, considering a single target SwA or the full OS (i.e., approach oriented to preserve the user privacy or a threat detection approach in the full system).

***Network**:* in this case, the hash analysis is different, while the content of the packets is exactly the same. Using the *Ratcliff/Obershelp* algorithm, the network captures are almost 100% equals. Some packets differ because of the different internals of the emulator to manage some connections (only regarding the number of flows, not its content, nor the sent/received payload). The latter includes, for example, network discovery requests, handshake packets, or Googlecast requests in the case of the mDT. The same results were obtained for the single target application network traffic analysis. This means that it is possible to reconstruct the user's network connection in any way with a target analysis of the interesting application, hence preserving its privacy. Conversely, a complete network analysis of the system traffic can be made. This analysis consents to know, for example, the visited websites, the timeframe, and the sent/received files (with also the possibility of extracting them).

***RAM**:* about the target SwA analysis, the hash is different because of the different offsets and acquisition time. Still, the content is exactly the same as computed with the *Ratcliff/Obershelp* algorithm, and the same data content, such as target strings of specific behaviors, can be identified. With the complete RAM analysis, the hash is different. But the content of data (e.g., files, processes, variables, etc.) extracted with Volatility is exactly the same. Therefore, the hash of the extracted files matches 100%. With this analysis, it is possible to dump the list of the running applications at a specific moment, how the OS manages them, and extract data from them (e.g., encryption keys, one-shot pictures sent in chats, hidden behaviors).

***Logical**:* for the target SwA, the content of its directory is completely the same between the mDT and the real device. In fact, the hash is the same, and the Ratcliff/Obershelp algorithm gives 100% of similarity. The same can be found for the other directories managed by the SwA, such as those in the *sdcard* to save sent or received multimedia.

For the full logical acquisition, because of the specific directory of the real device managed by the vendor or because of the specifics of the emulator, the hash is different, while the *Ratcliff/Obershelp* highlights a very high similarity. The logical analysis allows to monitor the creation and modification of specific files, both at the target SwA level or the whole filesystem.

***Physical Results**:* also in this case, due to the different files needed by the emulator and the real device, the hashes are different, but the content of the target behavior or applications is exactly the same. This analysis is very important as it allows to keep track, retrieve and reconstruct the deleted files.

VI. Research Challenges and Open Issues

The proposed MoLIFE methodology strictly depends on the technology used. For this reason, the current version of MoLIFE has some limitations related to its progress. Additionally, employing the MoLIFE system strictly depends on the country's regulations. In the case of Europe, for example, it must be GDPR compliant to respect privacy. Moreover, international law enforcement collaboration must be regulated, and the legislation of each country involved in the investigation must be respected. To be fully integrated into our daily lives, some standards on technology must be followed. At the time of writing this paper, some studies have been carried out on standardization, as highlighted by [45]. The MoLIFE system has some intrinsic limitations that do not allow use in every scenario. As an example, the system cannot monitor applications that detect the execution on an emulated environment, changing its behavior accordingly (MATE scenario). For example, some SwA cannot be monitored because of development restrictions that prevent it from being installed on devices with super-user access. It is the case of PayPal that cannot be installed from the Play Store, neither by `adb` if the device is rooted. Accordingly, some modifications to the SwA via reverse engineering techniques can be made to bypass such checks. Moreover, self-developed SwA by the company or other entities to be used for a specific use case is blocked during installation by the preventive stage on the hash matching in the database. This problem can be overcome by modifying the database, i.e., adding the hash of the SwA developed for the specific use case or adding the SwA in the official Store so that it is verified. This last option, sometimes for privacy restrictions, industrial secrets, or intellectual properties, cannot be adopted. Hence, modifying the database accordingly is the best solution. As highlighted in Section IV-B for the Android case study, some forensics methodologies and tools cannot be

[12]https://xlinux.nist.gov/dads/HTML/ratcliffObershelp.html

[13]https://docs.python.org/3/library/difflib.html#module-difflib

applied to iOS devices as they are less investigated due to the proprietary system's restrictions. However, the study by Hu *et al.* [46] discovered that SwA behaviors and vulnerabilities in Android are found in the same way in iOS devices. Thus, some features and detection mechanisms could remain unchanged, but more detailed investigations are needed. Regarding the SwA monitoring, the MoLIFE system is designed to monitor only one SwA of interest. Monitoring more than one SwA leads to an additional complex layer because every change in each SwA must be tracked. Tests must also be conducted to consider more than one SwA. At the time of presenting the MoLIFE methodology, no dataset is available to test the system in a real-case scenario. Additionally, no robust selected features have been publicly released, nor have accurate models been used to detect the presented behaviors. Accordingly, more research is needed in this direction to extend the use of the proposed methodology to other types of devices such as mobile devices based on iOS. Moreover, at this stage of development of MoLIFE, it is difficult to test the robustness to adversarial attacks. The estimation of the robustness is essential for better accuracy of the system, and reduce the possibility that detection is eluded. For example, robustness is essential in some critical cases such as the AI-based detection of violence (or other sensitive topics) via the real-time of chats. Finally, the security and protection of the MoLIFE system, with respect to risks related to the infrastructure where the mobile devices are used, should be duly analysed and addressed. For example, the protection of data over the communication channels (e.g., privacy preservation and data exfiltration, 5/6G communication security challenges), and the protection of the edge from attacks, and the protection of the mDT from unauthorized access and privilege escalation by the attacker, as highlighted in [47], should be better analyzed and addressed.

VII. Conclusion

This paper proposes a new mobile device forensics methodology (MoLIFE) based on new technologies. MoLIFE will analyze extracted interesting data in case of suspicious activity such as a cyber attack or a traditional crime to the involved owner. The methodology aims at overcoming the limitations of modern mobile devices that cannot be fully analyzed if no super-user privilege has been acquired before the rising alarm of a running potential threat. Moreover, it is safe not to previously acquire the super-user privilege on the physical devices because this could expose them to more vulnerabilities and attacks. In this case, a mDT with admin privileges can help monitor and analyze. The monitoring stage is controlled by AI algorithms, specifically trained to the presented analysis and robust to adversarial attacks, with the capability of explaining and reporting to the human forensics analyst the decision while keeping the real device as protected as possible. Moreover, this paper presents a case study on an Android system, showing how the forensic analysis of an mDT is the same as that of a real Android device. The proposed system has some limitations mainly related to the limitations of the current state of the technologies. The proposed system has the potentiality to evolve in agreement to the advance of the employed technologies.

Acknowledgments

This work was partially supported by Project SERICS (PE00000014r) under the NRRP MUR program funded by the EU - NGEU. This work was carried out while Silvia Lucia Sanna was enrolled in the Italian National Doctorate on Artificial Intelligence run by Sapienza University of Rome in collaboration with the University of Cagliari. This work was designed with the collaboration of Davide Maiorca and Leonardo Regano.

References

[1] H. Alatawi, K. Alenazi, S. Alshehri, S. Alshamakhi, M. Mustafa, and A. Aljaedi, "Mobile forensics: A review," in *2020 International Conference on Computing and Information Technology (ICCIT-1441)*, 2020, pp. 1–6.

[2] A. Sahani, "Android v/s ios – the unceasing battle," *International Journal of Computer Applications*, vol. 180, pp. 23–26, 12 2017.

[3] J. Howarth, "How many people own smartphones? (2024-2029)," *Exploding Topics*, 2025. [Online]. Available: https://explodingtopics.com/blog/smartphone-stats

[4] Data Bridge Market Research, "Global mobile computing devices market – industry trends and forecast to 2029," 2022. [Online]. Available: https://www.databridgemarketresearch.com/reports/global-mobile-computing-devices-market

[5] A. Al Hwaitat, M. Almaiah, A. Al-Zahrani, and O. Almomani, "Classification of cyber security threats on mobile devices and applications," 05 2021.

[6] B. Fakiha, "Unlocking digital evidence: Recent challenges and strategies in mobile device forensic analysis," *Journal of Internet Services and Information Security*, vol. 14, pp. 68–84, 08 2024.

[7] R. Ayers, S. Brothers, and W. Jansen, "Guidelines on mobile device forensics," 2014-05-15 00:05:00 2014.

[8] R. Cuomo, D. D'Agostino, and M. Ianulardo, "Mobile forensics: Repeatable and non-repeatable technical assessments," *Sensors*, 2022.

[9] A. Fukami, R. Stoykova, and Z. Geradts, "A new model for forensic data extraction from encrypted mobile devices," *Forensic Science International: Digital Investigation*, vol. 38, p. 301169, 2021.

[10] Q. Rossy, D. Décary-Hétu, O. Delémont, and M. Mulone, *The Routledge International Handbook of Forensic Intelligence and Criminology*, 01 2018.

[11] C. Hargreaves, A. Nelson, and E. Casey, "An abstract model for digital forensic analysis tools - a foundation for systematic error mitigation analysis," *Forensic Science International: Digital Investigation*, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2666281723001981

[12] M. Abdelrahman, E. Macatulad, B. Lei, M. Quintana, C. Miller, and F. Biljecki, "What is a digital twin anyway? deriving the definition for the built environment from over 15,000 scientific publications," *Building and Environment*, 2025.

[13] M. Grieves and J. Vickers, "Origins of the digital twin concept," *Florida Institute of Technology*, vol. 8, pp. 3–20, 2016.

[14] ——, "Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems," in *Transdisciplinary Perspectives on Complex Systems: New Findings and Approaches*, 2016.

[15] S. Boschert and R. Rosen, "Digital twin-the simulation aspect," in *Mechatronic Futures: Challenges and Solutions for Mechatronic Systems and Their Designers*, 2016.

[16] M. Liu, S. Fang, H. Dong, and C. Xu, "Review of digital twin about concepts, technologies, and industrial applications," *Journal of Manufacturing Systems*, vol. 58, 2021.

[17] A. Fuller, Z. Fan, C. Day, and C. Barlow, "Digital twin: Enabling technologies, challenges and open research," *IEEE Access*, 2020.

[18] P. Maddikunta, Q.-V. Pham, P. B, N. Deepa, K. Dev, T. Gadekallu, R. Ruby, and M. Liyanage, "Industry 5.0: A survey on enabling technologies and potential applications," *Journal of Industrial Information Integration*, vol. 26, 2022.

[19] J. Guo, D. Liu, R. Zhao, and Z. Li, "Wltdroid: Repackaging detection approach for android applications," *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 12432 LNCS, p. 579 – 591, 2020.

[20] D. Soi, S. L. Sanna, A. Liguori, M. Zuppelli, L. Regano, D. Maiorca, L. Caviglione, G. Manco, and G. Giacinto, "On the feasibility of android stegomalware: A detection study," 2025.

[21] X. Chen, C. Li, D. Wang, S. Wen, J. Zhang, S. Nepal, Y. Xiang, and K. Ren, "Android HIV: A Study of Repackaging Malware for Evading Machine-Learning Detection," *IEEE Transactions on Information Forensics and Security*, vol. 15, p. 987 – 1001, 2020.

[22] "Repacking ios applications." [Online]. Available: https://labs.withsecure.com/publications/repacking-and-resigning-ios-applications

[23] M. Ahammed, "Clone detection to prevent software piracy in android play store," *GSC Advanced Research and Reviews*, vol. 21, p. 108–131, 12 2024.

[24] M. E. Z. N. Kambar, A. Esmaeilzadeh, Y. Kim, and K. Taghva, "A survey on mobile malware detection methods using machine learning," 2022.

[25] H. Abubaker, F. Muchtar, S. Fattah, A. O. I. Elsayed, C. Salimun, H. Ismail, and F. Masud, "Machine learning approaches for malware classification in android platform: A review," *Journal of Advanced Research in Applied Sciences and Engineering Technology*, vol. 48, no. 1, p. 248 – 268, 2025.

[26] M. Saudi, M. Husainiamer, A. Ahmad, and M. Idris, "ios mobile malware analysis: a state-of-the-art," *Journal of Computer Virology and Hacking Techniques*, vol. 20, pp. 533–562, 05 2023.

[27] A. Demontis, M. Melis, B. Biggio, D. Maiorca, D. Arp, K. Rieck, I. Corona, G. Giacinto, and F. Roli, "Yes, machine learning can be more secure! a case study on android malware detection," *IEEE Trans. Dependable Secur. Comput.*, p. 711–724, Jul. 2019.

[28] A. Cimitile, F. Martinelli, and F. Mercaldo, "Machine learning meets ios malware: Identifying malicious applications on apple environment," vol. 2017-January, 2017.

[29] L. Minnei, G. Piras, A. Sotgiu, M. Pintor, A. Demontis, D. Maiorca, and B. Biggio, "An experimental analysis of semi-supervised learning for malware detection," 2025.

[30] B. Zhang, Z. Wen, X. Wei, and J. Ren, "Interdroid: An interpretable android malware detection method for conceptual drift," *Jisuanji Yanjiu yu Fazhan/Computer Research and Development*, 2021.

[31] D. Soi, A. Sanna, D. Maiorca, and G. Giacinto, "Enhancing android malware detection explainability through function call graph apis," *Journal of Information Security and Applications*, vol. 80, p. 103691, 2024.

[32] Y. Fratantonio, A. Bianchi, W. Robertson, E. Kirda, C. Kruegel, and G. Vigna, "Triggerscope: Towards detecting logic bombs in android applications," in *2016 IEEE Symposium on Security and Privacy (SP)*, 2016, pp. 377–396.

[33] J. Samhi and A. Bartel, "On the (in)effectiveness of static logic bomb detection for android apps," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 6, pp. 3822–3836, 2022.

[34] M. Kuštelega, R. Mekovec, and A. Shareef, "Privacy and security challenges of the digital twin: systematic literature review," *JUCS - Journal of Universal Computer Science*, vol. 30, pp. 1782–1806, 12 2024.

[35] S.-T. Sun, A. Cuadros, and K. Beznosov, "Android rooting: Methods, detection, and evasion." Association for Computing Machinery, 2015.

[36] I. A. Mohammad, A. O. Nasar, M. Alkhawaldeh, E.-U.-H. Qazi, and T. Zia, "Anti-forensic challenges in digital forensics investigations: An overview of techniques and tools," 2024.

[37] G. C. Kessler, "Anti-forensics and the digital investigator," 2007.

[38] S. Badhani and S. K. Muttoo, "Evading Android Anti-malware by Hiding Malicious Application Inside Images," *International Journal of System Assurance Engineering and Management*, vol. 9, pp. 482–493, 2018.

[39] M. Wong and D. Lie, "IntelliDroid: A Targeted Input Generator for the Dynamic Analysis of Android Malware," 2016.

[40] T. Kröll, S. Kleber, F. Kargl, M. Hollick, and J. Classen, "ARIstoteles – Dissecting Apple's Baseband Interface," *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 12972 LNCS, 2021.

[41] C. Xia, M. Paltenghi, J. Tian, M. Pradel, and L. Zhang, "Fuzz4ALL: Universal Fuzzing with Large Language Models," 2024.

[42] M. Alrammal, M. Naveed, S. Sallam, and G. Tsaramirsis, "A critical analysis on android vulnerabilities, malware, anti-malware and anti-malware bypassing," *Journal of Internet Technology*, vol. 23, no. 7, pp. 1651–1661, 2022, publisher Copyright: © 2022 Taiwan Academic Network Management Committee. All rights reserved.

[43] S. Khan, P. Krishnamoorthy, M. Goswami, F. M. Rakhimjonovna, S. A. Mohammed, and D. Menaga, "Quantum computing and its implications for cybersecurity: A comprehensive review of emerging threats and defenses," *Nanotechnology Perceptions*, 2024. [Online]. Available: https://nano-ntp.com/index.php/nano/article/view/2611

[44] K. Timms, "Byod must be met with a wider appreciation of the cybersecurity threat," *Computer Fraud and Security*, 2017.

[45] C. Alcaraz and J. Lopez, "Digital Twin Security: A Perspective on Efforts From Standardization Bodies," *IEEE Security and Privacy*.

[46] H. Hu, Y. Huang, Q. Chen, T. Y. Zhuo, and C. Chen, "A first look at on-device models in ios apps," 2023.

[47] C. Alcaraz and J. Lopez, "Digital Twin: A Comprehensive Survey of Security Threats," *IEEE Communications Surveys & Tutorials*, 2022.

Silvia Lucia Sanna received her BSc in Electrical, Electronic and Computer Engineering in 2019 with a Thesis on Brain Computer Interfaces. In 2022 she received her MSc in Computer Engineering, Cybersecurity and Artificial Intelligence with a thesis on Vulnerability Detection on Android Native Code. In 2023 she was Youth Ambassador for Women4Cyber Italy. Since November 2022 she is a PhD Student with the National PhD in Artificial Intelligence for Security and Cybersecurity. Her main research is focused on vulnerability and threat detection in mobile OS (Android) using AI and Digital Forensics.

Cristina Alcaraz is an Associate Professor in the Department of Computer Science at the University of Malaga. Her main research interests include security of cyber-physical systems, IIoT and digital twins; all of them applied to Industry 5.0, manufacturing and supply chain systems and smart grids.

Alessandro Sanna is a Post-Doc for the University of Cagliari, where is a member of the Pattern Recognition and Application Laboratory, and Abissi Srl since April 2021. Currently researching on Malware Detection and Threat Intelligence, his studies focus primarily on Living-off-the-Land Malware and the use of ML/DL for Malware and Threat Detection.

Giorgio Giacinto is a Professor of Computer Engineering at the University of Cagliari, Italy, where he serves as the coordinator of the M.Sc. degree in Computer Engineering, Cybersecurity and Artificial Intelligence. His research interests are in machine learning for malware analysis and detection, and he published more than 180 papers in international conferences and journals. He is the Editor in Chief of the "Security Engineering and Applications" section of the Journal of Cybersecurity and Privacy He is a member of the managing committees of the Cybersecurity National Lab and the Artificial Intelligence & Intelligent Systems Lab within the CINI consortium, Italy. He also represents the Cybersecurity National Lab within the European Cybersecurity Organization (ECSO) He is a Fellow of the IAPR (International Association for Pattern Recognition) and a Senior Member of the IEEE Computer Society and ACM.

Javier Lopez is Full Professor in the Department of Computer Science at the University of Malaga. His research activities are mainly focused on network security, security protocols and critical information infrastructures, and he leads a number of national and EU research projects in these areas.